Universidad de La Serena, Chile | FRC 2026

# Trabajo Práctico N° 2

## Flujo Integral de Machine Learning sobre Datos Tabulares:

*Exploración, Curación, Clustering, Regresión con Descenso por el Gradiente, Validación Cruzada e Inferencia Bayesiana MCMC*

**Profesor Titular:** Dr. Mariano Domínguez | **Entorno:** Python 3.10+ (Jupyter Notebook)

### Objetivos Pedagógicos y Alcance

El propósito fundamental de este trabajo práctico es integrar los fundamentos teóricos y metodológicos vistos en clase en un **flujo de trabajo completo, riguroso y reproducible de Ciencia de Datos y Machine Learning**, aplicado a un problema real perteneciente al área disciplinar o profesional de cada estudiante (Astronomía, Bioinformática, Geofísica, Economía, Finanzas, Medicina, Ingeniería, etc.).
Al finalizar este trabajo práctico, el estudiante será capaz de:

1. **Ingestar, explorar y curar** datos tabulares complejos mediante `pandas`, `scipy` y `seaborn`.
2. **Imputar y transformar** variables utilizando `scikit-learn`, justificando la selección de escaladores y esquemas de imputación según la presencia de valores atípicos y distribuciones subyacentes.
3. **Descubrir patrones latentes** aplicando algoritmos de agrupamiento no supervisado (*Clustering*) e interpretando el significado físico/conceptual de los conglomerados.
4. **Construir modelos predictivos supervisados** con expansión polinomial, optimizando los parámetros mediante **Descenso por el Gradiente** e investigando el impacto de la tasa de aprendizaje ($\eta$).
5. **Seleccionar la complejidad óptima** del modelo resolviendo el dilema sesgo-varianza (*Bias-Variance Trade-off*) mediante **Validación Cruzada ($k$-fold CV)**.
6. **Cuantificar la incertidumbre epistémica y observacional** aplicando **Inferencia Bayesiana** y muestreo estocástico mediante cadenas de Markov Monte Carlo (**MCMC**), analizando correlaciones y degeneraciones entre parámetros en diagramas de contornos (*corner plots*).

## 1. Directrices para la Elección del Dataset

Cada alumno (o grupo de trabajo, según la modalidad asignada) deberá seleccionar un **conjunto de datos tabular de su propio dominio profesional o de interés científico**. Para garantizar que el flujo de trabajo propuesto pueda desarrollarse en toda su profundidad, el dataset debe cumplir las siguientes directrices mínimas:

1. **Volumen y dimensionalidad:** Al menos $N \geq 500$ registros/observaciones y un mínimo de $P \geq 5$ variables numéricas (características continuas o discretas). Se valora positivamente la presencia de variables categóricas que requieran codificación.

2. **Estructura para aprendizaje supervisado:** Debe contener al menos una variable cuantitativa continua que tenga sentido modelar como **variable dependiente** ($y$) en función de una o más covariables predictoras ($X$).

3. **Realismo y calidad:** El dataset no debe estar previamente higienizado en su totalidad; debe presentar desafíos reales como datos faltantes (*missing values*), valores atípicos (*outliers*), correlaciones significativas o diferentes escalas de medición.

**Repositorios y Fuentes Sugeridas según Disciplina**

- **Astronomía y Ciencias Espaciales:** Catálogos de SDSS (galaxias, cuásares, fotometría/espectroscopía), archivo de exoplanetas de la NASA (*NASA Exoplanet Archive*), catálogos astrométricos de Gaia (DR2/DR3), curvas de luz TESS/Kepler.
- **Bioinformática, Medicina y Genómica:** Expresión génica (RNA-Seq/Microarrays en NCBI GEO / TCGA), indicadores clínicos de pacientes (UCI Machine Learning Repository, PhysioNet, Kaggle).
- **Economía, Finanzas y Negocios:** Indicadores macroeconómicos y de deuda (World Bank Open Data, FMI, FRED St. Louis), series financieras de activos y covariables de mercado (Yahoo Finance, Bloomberg Open Data).
- **Geociencias, Meteorología y Ambiente:** Datos de sensores satelitales (Copernicus, NOAA), estaciones meteorológicas, registros sismológicos de redes geofísicas.
- **Ingeniería y Procesos:** Mediciones de telemetría de turbinas, degradación de baterías, parámetros de calidad de producción.

## 2. Fase 1: Ingesta, Exploración (EDA), Curación y Preprocesamiento

**Fase 1: Ingesta de Datos, Análisis Exploratorio y Tratamiento con Scikit-Learn**

El objetivo de esta fase es comprender a fondo la naturaleza matemática y física de los datos, detectar anomalías y preparar una matriz de características limpia y estandarizada.

### 2.1. 1.1 Ingesta y Chequeo Estructural

- Realice la carga estructurada del archivo (`.csv`, `.parquet`, `.fits` o base relacional) utilizando `pandas`.
- Inspeccione las dimensiones del DataFrame (`shape`), los tipos de datos asociados a cada columna (`dtypes`), y genere un resumen numérico fundamental de posición y dispersión (`describe()`).
- Convierta variables categóricas o etiquetas de texto a formatos apropiados (ej. *One-Hot Encoding* con `pd.get_dummies` o `OneHotEncoder`, o *Ordinal Encoding*).

### 2.2. 1.2 Análisis Exploratorio de Datos (EDA) con Seaborn

Diseñe visualizaciones científicas auto-contenidas y publicables utilizando `seaborn` y `matplotlib`:

1. **Distribuciones Univariadas:** Grafique histogramas con estimación de densidad por kernel (*Kernel Density Estimation*, KDE) y diagramas de caja (*boxplots*) para inspeccionar asimetría (*skewness*) y curtosis.

2. **Estructura de Correlaciones:** Calcule la matriz de correlación lineal de Pearson y de correlación de rango de Spearman. Grafique un mapa de calor (`sns.heatmap`) anotado con una paleta de colores divergente centrada en cero. Identifique colinealidades severas entre predictoras.

3. **Diagrama de Pares (*Pairplot*):** Seleccione las 4 o 5 variables más representativas del problema y genere un `sns.pairplot`, coloreando opcionalmente por una variable de clase o subpoblación de interés.

## 2.3. 1.3 Diagnóstico y Curación de Datos Faltantes con Scikit-Learn

- Cuantifique la proporción y distribución de valores nulos (`NaN`, `None`, valores nulos centinela como $-999$ o 9999).
- Discuta la naturaleza del mecanismo de pérdida: ¿Faltantes completamente al azar (MCAR), faltantes al azar (MAR) o no al azar (MNAR)?
- Aplique esquemas de imputación fundados mediante `sklearn.impute`:
  - `SimpleImputer`: Imputación por media (para distribuciones simétricas) o mediana (robusta ante colas largas/outliers).
  - `KNNImputer`: Imputación multivariada preservando la estructura local de vecinos en el espacio de características.
  - `IterativeImputer` (MICE): Modelado multivariado condicional entre variables correlacionadas.
- Justifique explícitamente en el texto la técnica elegida para su conjunto de datos.

## 2.4. 1.4 Detección de Valores Atípicos (*Outliers*)

Analice la presencia de observaciones anómalas utilizando criterios matemáticos explícitos (e.g. criterio de Tukey basado en el Rango Intercuartílico: $\text{IQR} = Q_3 - Q_1$, definiendo cortes en $[Q_1 - 1.5\,\text{IQR}, Q_3 + 1.5\,\text{IQR}]$, distancias de Mahalanobis o modelos basados en densidad). Discuta si los outliers detectados representan errores observacionales de medición o fenómenos extremos genuinos del sistema bajo estudio.

## 2.5. 1.5 Transformaciones y Normalización / Escalado

Dado que tanto los algoritmos de agrupamiento basados en distancias (K-Means, GMM, DBSCAN) como los algoritmos de optimización por descenso por el gradiente son sumamente sensibles a las escalas de las variables:

$$z = \frac{x - \mu}{\sigma} \qquad (\texttt{StandardScaler}) \tag{1}$$

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \qquad (\texttt{MinMaxScaler}) \tag{2}$$

$$x_{\text{rob}} = \frac{x - \text{med}(x)}{\text{IQR}(x)} \qquad (\texttt{RobustScaler}) \tag{3}$$

- Si alguna variable presenta una asimetría muy pronunciada que abarca múltiples órdenes de magnitud (e.g. luminosidades, masas estelares, ingresos económicos), evalúe aplicar primero una transformación logarítmica $\log_{10}(x)$ o de potencia (`PowerTransformer` / Box-Cox / Yeo-Johnson).
- Aplique y compare el escalador seleccionado de `sklearn.preprocessing` (`StandardScaler` vs. `RobustScaler`). Explique por qué el escalado es un prerrequisito indispensable para los métodos de clustering y descenso por el gradiente.

# 3. Fase 2: Aprendizaje No Supervisado: Métodos de Agrupamiento

**Fase 2: Segmentación y Agrupamiento No Supervisado**

El objetivo es identificar subpoblaciones naturales o regímenes latentes en el espacio de características multidimensional, comparando al menos dos algoritmos con fundamentos matemáticos distintos.

## 3.1. 2.1 Formulación y Selección de Variables

Seleccione un subconjunto de al menos dos o tres variables físicas/disciplinares continuas que permitan caracterizar las distintas tipologías o regímenes del sistema (por ejemplo: color vs. magnitud en galaxias, expresión de genes marcadores, inflación vs. crecimiento en macroeconomía, etc.).

## 3.2. 2.2 Aplicación de Algoritmos de Clustering

Aplique obligatoriamente **K-Means** y al menos **un segundo algoritmo** entre los siguientes:

1. **Gaussian Mixture Models (GMM):** Agrupamiento probabilístico blando (*soft clustering*) mediante modelos de mezclas gaussianas con el algoritmo Expectation-Maximization (EM).
2. **DBSCAN:** Agrupamiento basado en densidad, capaz de descubrir conglomerados de morfología no esférica y rechazar ruido/outliers.
3. **Clustering Jerárquico Aglomerativo (*Agglomerative Hierarchical*):** Vinculación por mínima varianza (*Ward linkage*) o distancia media, explorando la jerarquía mediante dendrogramas.

## 3.3. 2.3 Determinación del Número Óptimo de Conglomerados

No fije el número de grupos arbitrariamente; fundamente la elección mediante métricas objetivas:

- **Método del Codo (*Elbow Method*) e Inercia:** Grafique la inercia (suma de distancias cuadráticas intra-cluster, WCSS) en función de $K \in [2, 10]$ e identifique el punto de inflexión.
- **Coeficiente de Silueta (*Silhouette Score*):** Evalúe la métrica de silueta $s \in [-1, 1]$ y grafique los diagramas de silueta por muestra para verificar la cohesión y separación de los grupos.
- **Criterios de Información (BIC y AIC para GMM):** Si utiliza GMM, evalúe el Criterio de Información Bayesiano ($BIC$) y de Akaike ($AIC$) variando $K$ y la estructura de covarianza (`spherical`, `diag`, `tied`, `full`):

$$\text{BIC} = -2\ln\mathcal{L} + p\ln(N), \qquad \text{AIC} = -2\ln\mathcal{L} + 2p \tag{4}$$

  donde $p$ es el número de parámetros libres y $\mathcal{L}$ es la máxima verosimilitud de la mezcla.
- **Ajuste de hiperparámetros en DBSCAN:** Si utiliza DBSCAN, grafique la curva de distancias a los $k$-vecinos más cercanos ($k$-NN distance plot) para fijar el radio de vecindad $\varepsilon$ y el umbral `min_samples`.

## 3.4. 2.4 Visualización e Interpretación de los Grupos

- Grafique la partición del espacio bidimensional o de componentes principales (PCA) coloreando las instancias según la etiqueta asignada por cada algoritmo. Si utiliza GMM, grafique las elipses de covarianza a $1\sigma$ y $2\sigma$ de cada componente gaussiana.

- **Discusión disciplinar:** Interprete qué significado físico, biológico o económico tiene cada cluster encontrado. ¿Reflejan poblaciones conocidas (e.g. secuencia roja vs. nube azul en galaxias, clases de riesgo en crédito, perfiles fenotípicos en genómica) o sugieren una subdivisión no trivial en los datos?

# 4. Fase 3: Aprendizaje Supervisado: Modelado Polinomial, Descenso por el Gradiente y Validación Cruzada

**Fase 3: Modelado de Regresión Polinomial y Optimización Numérica**

El objetivo es ajustar una relación no lineal entre variables mediante polinomios, optimizar los parámetros mediante Descenso por el Gradiente y seleccionar la complejidad óptima del modelo mediante Validación Cruzada para evitar sobreajuste.

## 4.1. 3.1 Definición del Problema de Regresión y Partición de Datos

- Seleccione una variable continua dependiente ($y$) y una o más variables predictoras continuas ($x$).
- Realice una partición reproducible de los datos en conjuntos de **Entrenamiento (Train)** y **Prueba (Test)** (ej. $80\% - 20\%$) mediante `train_test_split` fijando la semilla aleatoria (`random_state`). **El conjunto de Test debe quedar estrictamente aislado** hasta la evaluación final del modelo seleccionado.

## 4.2. 3.2 El Modelo Polinomial

Se propone aproximar la relación funcional mediante un modelo de regresión polinomial de grado $d$:

$$f(x; \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_d x^d = \sum_{j=0}^{d} w_j x^j = \boldsymbol{\phi}(x)^T \mathbf{w} \tag{5}$$

donde $\boldsymbol{\phi}(x) = [1, x, x^2, \ldots, x^d]^T$ representa el vector de características extendidas polinomiales (`sklearn.preprocessing.PolynomialFeatures`) y $\mathbf{w} = [w_0, w_1, \ldots, w_d]^T$ es el vector de coeficientes a determinar.

## 4.3. 3.3 Optimización de Parámetros por Descenso por el Gradiente

Defina la función de costo basada en el Error Cuadrático Medio (*Mean Squared Error*, MSE):

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^{N} (f(x_i; \mathbf{w}) - y_i)^2 = \frac{1}{2N} \|\boldsymbol{\Phi}\mathbf{w} - \mathbf{y}\|_2^2 \tag{6}$$

donde $\boldsymbol{\Phi} \in \mathbb{R}^{N \times (d+1)}$ es la matriz de diseño.

El gradiente analítico respecto al vector de parámetros $\mathbf{w}$ viene dado por:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \boldsymbol{\Phi}^T (\boldsymbol{\Phi}\mathbf{w} - \mathbf{y}) \tag{7}$$

La regla de actualización iterativa para cada época o iteración $t$ es:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \, \nabla_{\mathbf{w}} J(\mathbf{w}^{(t)}) \tag{8}$$

donde $\eta > 0$ es la **tasa de aprendizaje** (*learning rate*).

**Requerimientos de Implementación del Descenso por el Gradiente**

1. Implemente el algoritmo de Descenso por el Gradiente por lotes (*Batch Gradient Descent*) de manera vectorizada en Python/NumPy (o utilice `SGDRegressor` de `sklearn.linear_model`).
2. **Análisis de convergencia de $\eta$:** Experimente con al menos tres valores distintos de tasa de aprendizaje: uno demasiado pequeño (convergencia lenta), uno óptimo y uno demasiado grande (divergencia u oscilaciones inestables).
3. **Curva de Aprendizaje de Pérdida:** Grafique la función de costo $J(\mathbf{w})$ en función del número de iteraciones o épocas para documentar la convergencia matemática.

Listing 1: Esqueleto orientativo de Descenso por el Gradiente por lotes vectorizado en NumPy.

```python
import numpy as np

def batch_gradient_descent(Phi, y, eta=0.01, n_epochs=2000, tol=1e-6):
    """
    Optimiza w mediante Batch Gradient Descent sobre la funcion de costo MSE.
    Phi: matriz de diseno (N, d+1)
    y: vector objetivo continuo (N,)
    """
    N, p = Phi.shape
    w = np.zeros(p)  # Inicializacion de coeficientes en cero
    loss_history = []

    for epoch in range(n_epochs):
        residuals = Phi @ w - y
        loss = (1.0 / (2.0 * N)) * np.sum(residuals**2)
        loss_history.append(loss)

        # Calculo analitico exacto del gradiente
        grad = (1.0 / N) * (Phi.T @ residuals)
        w_new = w - eta * grad

        # Criterio de parada por convergencia de parametros
        if np.linalg.norm(w_new - w) < tol:
            print(f"Convergencia alcanzada exitosamente en la epoca {epoch}")
            break
        w = w_new

    return w, np.array(loss_history)
```

## 4.4. 3.4 Optimización de Hiperparámetros mediante Validación Cruzada ($k$-Fold CV)

El grado polinomial $d$ actúa como un **hiperparámetro de complejidad**. Un grado insuficiente ($d = 1$) provocará subajuste (*underfitting*, alto sesgo), mientras que un grado excesivo ($d \gg 1$) inducirá sobreajuste (*overfitting*, alta varianza):

1. Implemente un esquema de **Validación Cruzada con $k$ pliegues ($k$-Fold Cross Validation, con $k = 5$ o 10)** explorando una rejilla de grados polinomiales $d \in \{1, 2, 3, 4, 5, 6, \ldots, D_{\max}\}$.
2. (Opcional recomendado): Si incluye un término de regularización Ridge ($L_2$) o Lasso ($L_1$), optimice simultáneamente la fuerza de regularización $\alpha$ utilizando `GridSearchCV` o un bucle estructurado con `KFold`.
3. **Curva de Validación (*Validation Curve*):** Grafique el error cuadrático medio de entrenamiento ($MSE_{\text{train}}$) y el error promedio de validación cruzada ($MSE_{\text{val}}$) junto con sus barras de error estándar en función del grado polinomial $d$.

4. **Selección de la complejidad óptima ($d^*$):** Identifique el grado que minimiza el error de validación sin incrementar la varianza.

5. **Evaluación final en Test:** Ajuste el modelo de grado óptimo $d^*$ con todo el conjunto de entrenamiento y evalúe su generalización de forma no sesgada sobre el conjunto de Test retenido, reportando las métricas:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}, \qquad \text{RMSE} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (y_i - \hat{y}_i)^2}, \qquad \text{MAE} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} |y_i - \hat{y}_i| \quad (9)$$

# 5. Fase 4: Inferencia Bayesiana: Muestreo MCMC de los Parámetros del Modelo Óptimo

**Fase 4: Inferencia Bayesiana con Cadenas de Markov Monte Carlo (MCMC)**

El objetivo es superar las limitaciones de la estimación puntual del descenso por el gradiente, obteniendo la distribución de probabilidad a posteriori completa (*Posterior PDF*) de los parámetros del polinomio óptimo y cuantificando las incertidumbres y covarianzas entre coeficientes.

## 5.1. 5.1 Formulación Probabilística Bayesiana

Considere el modelo de regresión polinomial con la complejidad óptima $d^*$ seleccionada en la Fase 3. Supondremos que cada observación $y_i$ se genera a partir de la función polinomial subyacente más un término de ruido gaussiano estocástico con dispersión $\sigma$:

$$y_i = f(x_i; \mathbf{w}) + \epsilon_i = \sum_{j=0}^{d^*} w_j x_i^j + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (10)$$

El vector completo de parámetros a inferir es $\boldsymbol{\theta} = [w_0, w_1, \ldots, w_{d^*}, \sigma]^T \in \mathbb{R}^{d^*+2}$ (o $\ln \sigma$).

## 5.2. 5.2 Funciones de Verosimilitud, Priors y Posterior

Bajo la hipótesis de observaciones independientes e idénticamente distribuidas, la función de **Log-Verosimilitud** ($\ln \mathcal{L}$) es:

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - f(x_i; \mathbf{w}))^2 \quad (11)$$

Si los datos cuentan con incertidumbres de medición heterocedásticas reportadas $\sigma_{y,i}$, la verosimilitud se generaliza como:

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{2} \sum_{i=1}^{N} \left[ \frac{(y_i - f(x_i; \mathbf{w}))^2}{\sigma_{y,i}^2 + \sigma_{\text{int}}^2} + \ln\left(2\pi(\sigma_{y,i}^2 + \sigma_{\text{int}}^2)\right) \right] \quad (12)$$

donde $\sigma_{\text{int}}$ representa la dispersión intrínseca del sistema.

- **Distribuciones a Priori ($p(\boldsymbol{\theta})$):** Seleccione y justifique las distribuciones a priori de los parámetros:
  - Priors planos o débilmente informativos uniformes para los coeficientes: $w_j \sim \mathcal{U}(w_{j,\min}, w_{j,\max})$ o gaussianos amplios $w_j \sim \mathcal{N}(0, \sigma_w^2)$.

  - Prior de escala de Jeffreys para el ruido: $p(\sigma) \propto 1/\sigma \iff p(\ln \sigma) \propto \text{const}$, o un prior Half-Cauchy / Half-Normal positivo.

- **Log-Posterior:** Aplicando el Teorema de Bayes:

$$\ln p(\boldsymbol{\theta} \mid \mathcal{D}) = \ln \mathcal{L}(\boldsymbol{\theta}) + \ln p(\boldsymbol{\theta}) + \text{constante} \tag{13}$$

### 5.3. 5.3 Muestreo de la Posterior con MCMC (`emcee` o `PyMC`)

- Utilice la biblioteca `emcee` (*Affine-Invariant Ensemble Sampler*) o alternativamente `PyMC` / `Stan`.
- Inicialice un ensamble de al menos $N_{\text{walkers}} \geq 2 \times \dim(\boldsymbol{\theta})$ caminantes (e.g. 32 a 64 caminantes) en una pequeña bola gaussiana alrededor de los valores óptimos obtenidos por Descenso por el Gradiente.
- Ejecute la cadena durante un número representativo de pasos (e.g. $N_{\text{steps}} \geq 2000$).

### 5.4. 5.4 Diagnóstico de Convergencia

1. **Gráfico de Trazas (*Trace Plots*):** Grafique la evolución temporal de cada parámetro $\theta_k$ en función de la iteración para todos los caminantes. Verifique que las trayectorias se hayan mezclado de manera homogénea (*well-mixed*).
2. **Período de Calentamiento (*Burn-in*):** Descarte las primeras muestras de la cadena (típicamente las primeras 300 a 500 iteraciones) antes de que el ensamble alcance el estado estacionario.
3. **Tiempo de Autocorrelación Integrada ($\tau$):** Estime el tiempo de autocorrelación $\tau_k$ para cada parámetro y verifique la condición de confiabilidad estándar: $N_{\text{steps}} \geq 50\,\tau$.

### 5.5. 5.5 Análisis de las Distribuciones Posteriores y Propagación de Incertidumbre

1. **Diagrama de Esquinas (*Corner Plot*):** Con la librería `corner`, genere la matriz de distribuciones posteriores marginales univariadas y contornos de densidad conjunta bidimensional a $1\sigma$ (39.3%), $2\sigma$ (86.5%) y $3\sigma$.
2. **Parámetros y Credibilidad:** Calcule los estimadores de tendencia central (mediana o moda posterior / MAP) y los intervalos de credibilidad del 68% y 95% (percentiles 16%, 50% y 84%):

$$\theta_k = \hat{\theta}_{k,\text{med}}{}^{+\sigma_+}_{-\sigma_-} \tag{14}$$

3. **Contraste con Descenso por el Gradiente:** Compare en una tabla estructurada los valores de los parámetros obtenidos por Descenso por el Gradiente (estimador puntual tipo Maximum Likelihood) frente a la mediana e intervalos de credibilidad obtenidos por MCMC. Discuta la coherencia entre ambos métodos.
4. **Propagación de Incertidumbre sobre el Ajuste:** Extraiga 100 realizaciones aleatorias de $\mathbf{w}$ a partir de la cadena MCMC ya convergida y grafique las curvas polinomiales resultantes sobre los datos de dispersión originales, delimitando las **bandas de credibilidad del 68% y 95%** del ajuste.

Listing 2: Esqueleto orientativo de muestreo Bayesiano MCMC con `emcee` y visualización con `corner`.

```
import emcee
import corner
import numpy as np
import matplotlib.pyplot as plt

# 1. Definicion de Log-Likelihood, Log-Prior y Log-Probability
```

```python
def log_likelihood(theta, x, y, yerr=None):
    w, log_sigma = theta[:-1], theta[-1]
    sigma = np.exp(log_sigma)
    model = np.polyval(w[::-1], x)  # w_0 + w_1*x + ... + w_d*x^d
    if yerr is None:
        sigma2 = sigma**2
    else:
        sigma2 = yerr**2 + sigma**2
    return -0.5 * np.sum((y - model)**2 / sigma2 + np.log(2 * np.pi * sigma2))

def log_prior(theta):
    w, log_sigma = theta[:-1], theta[-1]
    # Priors gaussianos amplios para w y uniforme para log(sigma)
    if -10.0 < log_sigma < 5.0 and np.all(np.abs(w) < 1e3):
        return -0.5 * np.sum((w / 50.0)**2)
    return -np.inf

def log_probability(theta, x, y, yerr=None):
    lp = log_prior(theta)
    if not np.isfinite(lp):
        return -np.inf
    return lp + log_likelihood(theta, x, y, yerr)

# 2. Inicializacion de caminantes (walkers) alrededor del optimo
ndim = len(w_optimal) + 1  # coeficientes polinomiales + log_sigma
nwalkers = 32
initial_pos = [np.append(w_optimal, np.log(np.std(residuals))) + 1e-4 *
    np.random.randn(ndim)
               for _ in range(nwalkers)]

# 3. Muestreo con EnsembleSampler
sampler = emcee.EnsembleSampler(nwalkers, ndim, log_probability, args=(x_train,
    y_train))
sampler.run_mcmc(initial_pos, 2500, progress=True)

# 4. Descarte de burn-in y extraccion de cadenas aplanadas
tau = sampler.get_autocorr_time(quiet=True)
burn_in = int(2 * np.max(tau)) if np.all(np.isfinite(tau)) else 500
flat_samples = sampler.get_chain(discard=burn_in, thin=15, flat=True)

# 5. Generacion del Corner Plot con cuantiles
labels = [f"$w_{i}$" for i in range(len(w_optimal))] + [r"$\ln\sigma$"]
fig = corner.corner(flat_samples, labels=labels, quantiles=[0.16, 0.50, 0.84],
                    show_titles=True, title_fmt=".3f")
plt.show()
```

## 6. Formato de Entrega y Estructura del Trabajo

El trabajo práctico se presentará a través del repositorio institucional o plataforma de entrega en los siguientes formatos:

1. **Jupyter Notebook reproducible (`.ipynb`):**
    - Código organizado en módulos o celdas lógicas, con comentarios explicativos claros.
    - Semillas aleatorias fijadas (`random_state=42` o equivalente) para asegurar la exacta reproducibilidad de los experimentos.
    - Todas las figuras deben ser auto-contenidas: títulos informativos, nombres de ejes con sus respectivas unidades físicas/conceptuales y leyendas legibles.

2. **Informe Técnico / Reporte Académico (PDF de 6 a 10 páginas):** Estructurado bajo formato científico estándar:
   - **1. Introducción y Contexto Disciplinar:** Descripción del problema científico o industrial, relevancia de los datos y preguntas a responder.
   - **2. Exploración, Curación y Preprocesamiento:** Resumen del EDA, justificación de imputación y normalización.
   - **3. Análisis de Agrupamiento (Clustering):** Comparación de algoritmos, métricas de número de clusters e interpretación de regímenes.
   - **4. Modelado Polinomial y Validación Cruzada:** Descenso por el gradiente, curvas de pérdida, análisis de sesgo-varianza y selección de $d^*$.
   - **5. Inferencia Bayesiana MCMC:** Definición de verosimilitud y priors, diagnósticos de convergencia, corner plot e interpretación de la incertidumbre.
   - **6. Conclusiones y Discusión:** Síntesis de hallazgos disciplinarios, limitaciones del modelo y perspectivas futuras.

## 7. Rúbrica de Evaluación Ponderada

La calificación final del Trabajo Práctico N° 2 se regirá por la siguiente escala ponderada de 100 puntos totales:

| Componente / Fase | Criterios de Evaluación y Rigor Metodológico | Puntaje |
|---|---|---|
| **1. Dataset, Ingesta & EDA** | Selección adecuada de dataset con complejidad real ($N \geq 500$). Análisis exploratorio riguroso con distribuciones, correlaciones y pairplots en Seaborn. Interpretación conceptual de variables. | **15 pts** |
| **2. Curación, Imputación & Normalización** | Diagnóstico de nulos. Implementación y justificación técnica de imputación en Scikit-Learn (`SimpleImputer` / `KNNImputer`). Detección de outliers y fundamentación del escalador seleccionado. | **15 pts** |
| **3. Clustering & Métodos No Supervisados** | Aplicación y comparación de K-Means y un segundo método (GMM/DBSCAN/Jerárquico). Selección fundada de hiperparámetros (Codo, Silueta, BIC/AIC). Interpretación disciplinar profunda de los grupos. | **20 pts** |
| **4. Regresión Polinomial, Gradiente & CV** | Ajuste polinomial con optimización por Descenso por el Gradiente. Estudio de convergencia y tasa de aprendizaje $\eta$. Validación cruzada ($k$-fold) para optimizar el grado $d^*$. Métricas en Test ($R^2$, RMSE). | **25 pts** |
| **5. Inferencia Bayesiana & MCMC** | Formulación probabilística rigurosa (Verosimilitud, Priors, Log-Posterior). Diagnósticos de convergencia (trazas, autocorrelación, burn-in). Corner plot, intervalos de credibilidad y contraste con el gradiente. | **20 pts** |
| **6. Calidad del Informe & Reproducibilidad** | Claridad expositiva, redacción científica, código limpio y ejecutable de punta a punta sin errores, reproducibilidad de figuras y resultados. | **5 pts** |
| **TOTAL** | | **100 pts** |

Cuadro 1: Rúbrica detallada de calificación para el Trabajo Práctico N° 2.

## 8. Bibliografía y Referencias Recomendadas

1. **Ivezić, Ž., Connolly, A. J., VanderPlas, J. T., & Gray, A.** (2014). *Statistics, Data Mining, and Machine Learning in Astronomy: A Practical Python Guide for the Analysis of Survey Data.* Princeton University Press.

2. **Hastie, T., Tibshirani, R., & Friedman, J.** (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

3. **Foreman-Mackey, D., Hogg, D. W., Lang, D., & Goodman, J.** (2013). *`emcee`: The MCMC Hammer*. Publications of the Astronomical Society of the Pacific, 125(925), 306.

4. **Hogg, D. W., Bovy, J., & Lang, D.** (2010). *Data analysis recipes: Fitting a model to data.* arXiv preprint arXiv:1008.4686.

5. **Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B.** (2013). *Bayesian Data Analysis* (3rd ed.). Chapman and Hall/CRC.

6. **Scikit-Learn Documentation:** `https://scikit-learn.org/stable/`

7. **Emcee Documentation:** `https://emcee.readthedocs.io/en/stable/`